

MOTILAL NEHRU NATIONAL INSTITUTE OF TECHNOLOGY ALLAHABAD

Prayagraj, Uttar Pradesh – 211004, India

Department of Electronics and Communication Engineering

A PROJECT REPORT

on

SPI EEPROM MOTOR-PROFILE CONTROLLER

using LPC2138 Microcontroller, 25LC256 SPI EEPROM and L293D Motor Driver

Submitted to

Dr. Shishir Maheshwari

Department of Electronics and Communication Engineering
Motilal Nehru National Institute of Technology Allahabad

Submitted by: Team 1

Academic Session 2026–2027

PROJECT DETAILS

Project	SPI EEPROM Motor-Profile Controller
Microcontroller	NXP LPC2138 / LPC2148 (ARM7TDMI-S)
Profile memory	Microchip 25LC256 SPI EEPROM, six motor profiles
Motor stage	L293D H-bridge driver and 12 V DC motor
Display	16×2 LCD (LM016L) and status LED
Firmware / tools	Embedded C, Keil µVision (ARM compiler)
Simulation	Proteus 8 Professional

SUBMITTED BY

Team 1, Department of Electronics and Communication Engineering

S. No.	Name of Student	Roll Number
1	Anurag Nirmal	20244031
2	Arin Saraf	20244034
3	Arun Kumar	20244038
4	Ashish Kumar Kashyap	20244043
5	Ashish Pandey	20244044

UNDER THE GUIDANCE OF

Dr. Shishir Maheshwari
Department of Electronics and Communication Engineering
Motilal Nehru National Institute of Technology Allahabad, Prayagraj

Academic Session 2026–2027

ABSTRACT

This project is an **SPI EEPROM Motor-Profile Controller** made with the NXP **LPC2138** microcontroller. A motor profile is a list of steps; each step gives the direction, speed, ramp time and run time of a DC motor. Six profiles that copy everyday machines — a demonstration, washing-machine wash and spin, a gate, a fan and a mixer — are stored in a **25LC256** EEPROM chip, which the microcontroller reads over **SPI**. Each profile is stored in its own page with a checksum, and any missing or damaged profile is repaired automatically.

The motor is driven through an **L293D** motor driver. Its speed is set with 100 Hz PWM, it changes speed smoothly, and it always pauses for 200 ms before changing direction. A **16×2 LCD** shows the profile, direction, speed and step, and an **LED** blinks differently for forward, reverse, stop and speed change. All timing comes from one timer, so the sequence never drifts. The program was written in C in Keil µVision and tested in Proteus, where the motor followed every profile and the EEPROM contents matched the program exactly.

Keywords: LPC2138, ARM7, SPI, EEPROM, 25LC256, L293D, PWM, DC motor, LCD, Keil, Proteus.

ACKNOWLEDGEMENT

We express our sincere gratitude to **Dr. Shishir Maheshwari**, Department of Electronics and Communication Engineering, Motilal Nehru National Institute of Technology Allahabad, for his guidance and for giving us the opportunity to work on this project.

We also thank the Department of Electronics and Communication Engineering, MNNIT Allahabad, for providing the academic environment and resources that made this work possible. Finally, we thank our classmates and families for their encouragement throughout the project.

Team 1
Department of ECE, MNNIT Allahabad

TABLE OF CONTENTS

ABSTRACT ..... **ii**
ACKNOWLEDGEMENT ..... **ii**
1. INTRODUCTION **1**
1.1 Project Overview 1
1.2 Why This Project 1
1.3 Tools Used 1
2. OBJECTIVES **2**
3. BASIC CONCEPTS **3**
3.1 LPC2138 Microcontroller 3
3.2 SPI Communication 4
3.3 25LC256 EEPROM 4
3.4 Motor Control with the L293D and PWM 4
3.5 16×2 LCD 5
4. HARDWARE DESIGN **6**
4.1 Block Diagram 6
4.2 Components and Connections 6
4.3 Proteus Circuit 7
5. MOTOR PROFILES AND EEPROM STORAGE **8**
5.1 The Six Profiles 8
5.2 How a Profile Is Stored 8
6. HOW THE SYSTEM WORKS **10**
6.1 Step-by-Step Operation 10
6.2 Exact Timing 10
6.3 Main Calculations 10
6.4 LED Patterns and LCD Screen 10
7. SOFTWARE **12**
7.1 Building the Program in Keil 12
7.2 Program Structure 12
7.3 Error Handling 13
8. SOURCE CODE **14**
9. RESULTS **22**
9.1 Program Build and EEPROM Contents 22
9.2 Profiles Running in Proteus 22
9.3 LED and LCD at Different Times 23
9.4 Timing Check 25
9.5 Problems Faced and Solutions 26
10. APPLICATIONS AND FUTURE SCOPE **27**
10.1 Applications 27
10.2 Future Scope 27
11. CONCLUSION **28**
REFERENCES **29**

LIST OF FIGURES

Figure 3.1: Clock settings used in the project 3
Figure 3.2: LPC2138 memory map 4
Figure 4.1: Block diagram of the system 6
Figure 4.2: Complete circuit in Proteus 7
Figure 5.1: EEPROM page map and the layout of one page 8
Figure 6.1: LED patterns over two seconds 11
Figure 7.1: Flowchart of the program 13
Figure 9.1: Keil µVision build output 22
Figure 9.2: EEPROM contents in Proteus after the first run 22
Figure 9.3: DEMO running forward (342 RPM) and WASH on its third repeat (278 RPM) 22
Figure 9.4: SPIN speeding up to 100 % (404 RPM) and GATE opening (371 RPM) 23
Figure 9.5: MIXER slowing down after a pulse (268 RPM) 23

LIST OF TABLES

Table 1.1: Tools used 1
Table 3.1: LPC2138 parts used in this project 3
Table 3.2: EEPROM commands used 4
Table 3.3: Motor direction control 4
Table 4.1: Components 6
Table 4.2: Pin connections 6
Table 5.1: Motor profiles 8
Table 5.2: Contents of one page 8
Table 6.1: Main calculations 10

Table 6.2: LED patterns....10
Table 7.1: Main functions....12
Table 7.2: Error codes....13
Table 9.1: Motor speed seen in Proteus....23
Table 9.2: LED and LCD status during the full sequence....24
Table 9.3: Test results....25
Table 9.4: Problems and solutions....26

1. INTRODUCTION

1.1 Project Overview

Many machines at home run a motor in a fixed pattern. A washing machine turns the drum forward and backward and then spins it fast; a gate opens, waits and closes; a fan changes its speed. In this project one controller can run all of these patterns.

Each pattern is called a **motor profile**. A profile is a short list of steps, and every step says four things: the **direction**, the **speed**, how long to take to reach that speed (**ramp time**) and how long to keep it (**run time**). Six profiles are stored in a **25LC256 EEPROM** chip. When the power is switched on, the **LPC2138** microcontroller reads the profiles, checks that they are correct and runs them one after another on a DC motor through an **L293D** motor driver. A **16×2 LCD** shows what the motor is doing, and an **LED** blinks in a different way for forward, reverse, stop and speed change.

1.2 Why This Project

- **One controller, many machines:** the behaviour comes from data stored in the EEPROM, not from different programs.
- **Settings are not lost:** the EEPROM keeps the profiles when the power is off.
- **Easy to see:** the LCD and LED show the controller's state at every moment.
- **Safe for the motor:** the speed changes slowly, and the motor is always switched off for a short time before it changes direction.

1.3 Tools Used

Table 1.1: *Tools used*

Tool	Use
Keil µVision (ARM compiler)	Writing and compiling the C program, making the .hex file
Proteus 8 Professional	Drawing the circuit and simulating it
Windows PC	Running the tools

2. OBJECTIVES

1. Connect a 25LC256 EEPROM to the LPC2138 over SPI and read and write it.
2. Store six motor profiles in the EEPROM in a safe format and repair them automatically if they are missing or damaged.
3. Make profiles that copy real machines: washing-machine wash and spin, gate, fan, mixer and a demonstration profile.
4. Control the speed and direction of a DC motor with an L293D using PWM, with smooth speed changes and a pause before reversing.
5. Show the state of the system on a 16×2 LCD and with LED blink patterns.
6. Test the complete system in Proteus.

3. BASIC CONCEPTS

3.1 LPC2138 Microcontroller

The LPC2138 from NXP is a 32-bit microcontroller with an **ARM7TDMI-S** processor. It has 512 KB of flash memory for the program, 32 KB of RAM for variables, and many built-in peripherals: timers, SPI, UART, I²C, ADC, DAC and PWM. It works from a single 3.3 V supply. The LPC2148 is almost the same chip with USB added, so the same program works on both.

Table 3.1: *LPC2138 parts used in this project*

Part of the chip	What it does here
ARM7 processor at 12 MHz	Runs the program
GPIO (general-purpose pins)	Drives the LED, LCD, motor direction and EEPROM chip-select; reads the button
Timer1	Counts every 10 µs — the clock for all timing
SPI0	Talks to the EEPROM
Pin select (PINSEL)	Chooses which pins are SPI and which are GPIO
Clock control (PLL, VPBDIV)	Sets the processor clock to 12 MHz and the peripheral clock to 3 MHz

How the clock is set (Figure 3.1). The 12 MHz crystal gives the processor clock directly, because the PLL (a clock multiplier) is switched off in the program. The peripheral clock is one quarter of it, 3 MHz. Timer1 divides this by 30 to count every 10 µs, and SPI divides it by 8 to give a 375 kHz SPI clock.

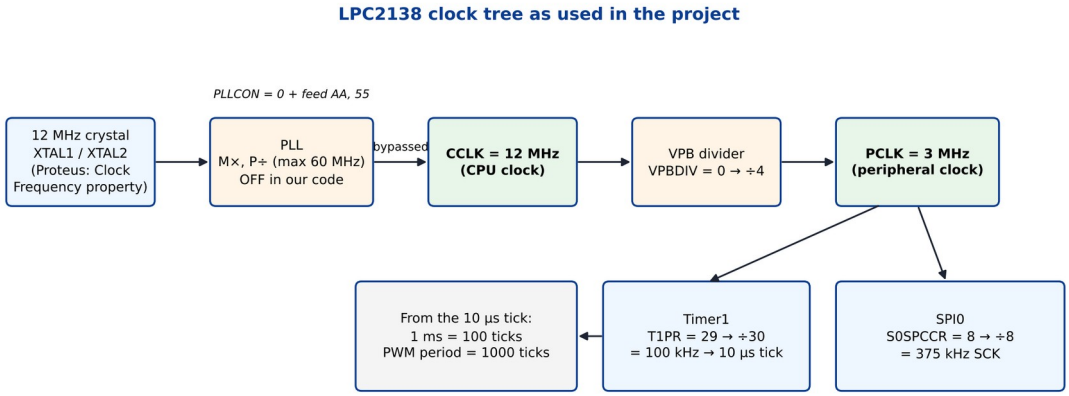


Figure 3.1: *Clock settings used in the project*

Memory map (Figure 3.2). Every part of the chip has a fixed address. The program is in flash starting at address 0, the variables are in RAM starting at 0x4000 0000, and each peripheral's control registers are at fixed addresses from 0xE000 0000. The program controls a peripheral by reading and writing these addresses.

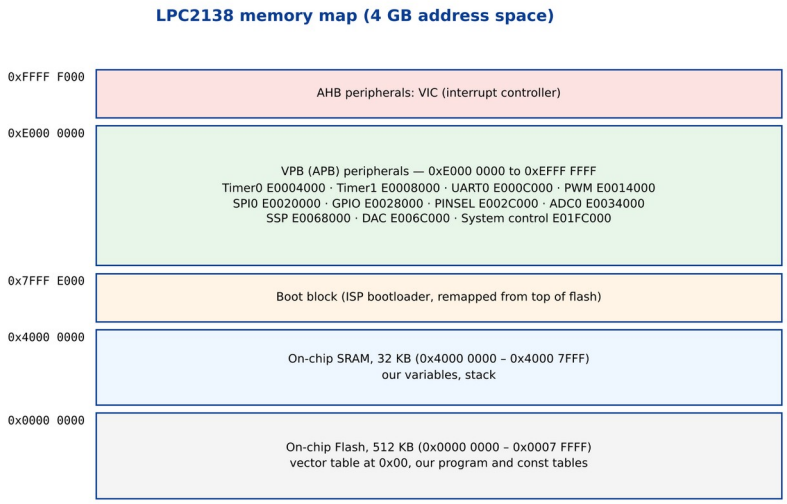


Figure 3.2: *LPC2138 memory map*

3.2 SPI Communication

SPI is a simple way for two chips to exchange data using four wires: a clock (**SCK**) from the master, data out of the master (**MOSI**), data into the master (**MISO**) and a chip-select (**CS**) that tells the slave to listen. Every time the master sends one byte it also receives one byte. The LPC2138 is the master and the EEPROM is the slave. We use **mode 0** (clock is low when idle, data is read on the rising edge) at 375 kHz.

3.3 25LC256 EEPROM

The 25LC256 stores 32 KB of data that is kept when the power is off. Memory is divided into **pages of 64 bytes**. Before every write, writing must be enabled with a command, and after a write the chip is busy for up to 5 ms.

Table 3.2: *EEPROM commands used*

Command	Code	Meaning
WREN	0x06	Allow writing
RDSR	0x05	Read the status (busy? write allowed?)
READ	0x03	Read data from an address
WRITE	0x02	Write up to 64 bytes inside one page

3.4 Motor Control with the L293D and PWM

A microcontroller pin cannot give the current a motor needs, so the **L293D** driver is used. Its two inputs IN1 and IN2 choose the direction, and its enable pin EN1 switches the motor on and off. **PWM** (pulse-width modulation) switches EN1 on and off quickly — 100 times a second here. The percentage of time it is on (the duty cycle) sets the average voltage and so the speed.

Table 3.3: *Motor direction control*

EN1	IN1	IN2	Motor
PWM	1	0	Forward
PWM	0	1	Reverse
any	0	0	Off (coasts)
any	1	1	Brake — never used

3.5 16×2 LCD

The LCD shows 2 lines of 16 characters. It is used in **4-bit mode**: each byte is sent in two halves on four data lines, which saves pins. The RS pin says whether the byte is a command or a character, and a pulse on the E pin makes the LCD read it.

4. HARDWARE DESIGN

4.1 Block Diagram

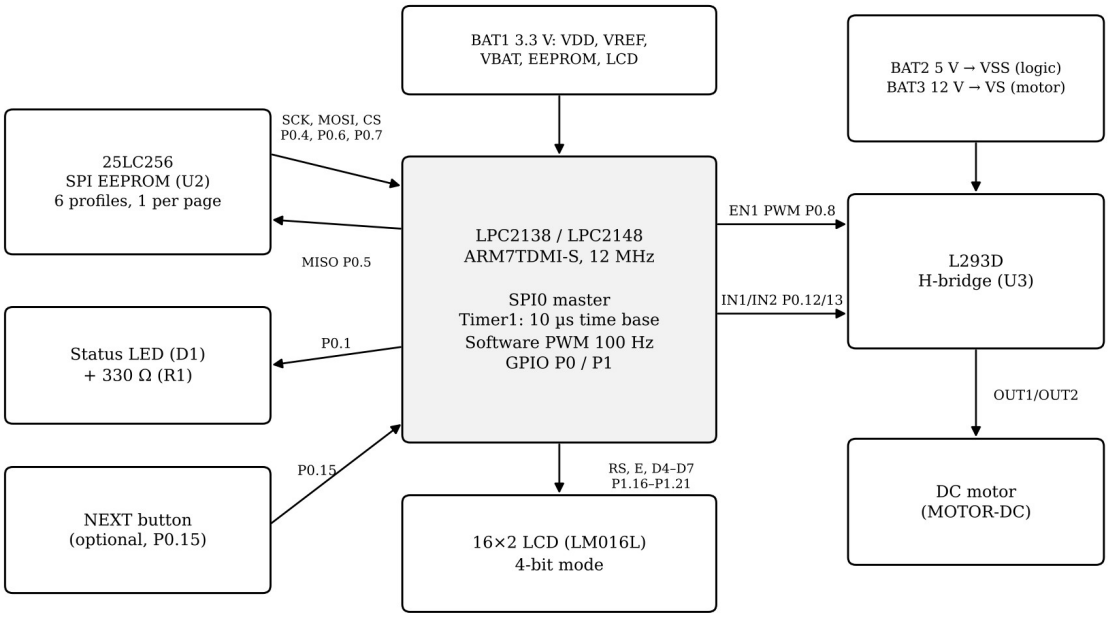


Figure 4.1: *Block diagram of the system*

4.2 Components and Connections

Table 4.1: *Components*

Part	Component	Job
U1	LPC2138 (or LPC2148)	Controller
U2	25LC256 EEPROM	Stores the six profiles
U3	L293D	Motor driver
LCD1	16×2 LCD (LM016L)	Shows the status
D1, R1	LED + 330 Ω resistor	Status light
M1	DC motor	The load
BAT1, BAT2, BAT3	3.3 V, 5 V, 12 V	Power for the controller, driver logic and motor
S1 (optional)	Push-button + 10 kΩ	NEXT profile

Table 4.2: *Pin connections*

LPC2138 pin	Used for	Connected to
P0.4, P0.5, P0.6	SPI clock, data in, data out	EEPROM SCK, SO, SI
P0.7	EEPROM chip-select	EEPROM CS
P0.8	PWM (speed)	L293D EN1
P0.12, P0.13	Direction	L293D IN1, IN2
P0.1	LED	330 Ω → LED → GND
P0.15	NEXT button	Button to GND, 10 kΩ to 3.3 V
P1.16, P1.17	LCD RS, E	LCD pins 4, 6

LPC2138 pin	Used for	Connected to
P1.18 – P1.21	LCD data D4 – D7	LCD pins 11 – 14

Power: 3.3 V for the LPC2138, EEPROM and LCD; 5 V for the L293D logic; 12 V for the motor. All grounds are joined. In Proteus the crystal is not drawn — the chip's *Clock Frequency* property is set to **12 MHz**. A real board would also need a 12 MHz crystal with two 22 pF capacitors, a reset circuit and decoupling capacitors.

4.3 Proteus Circuit

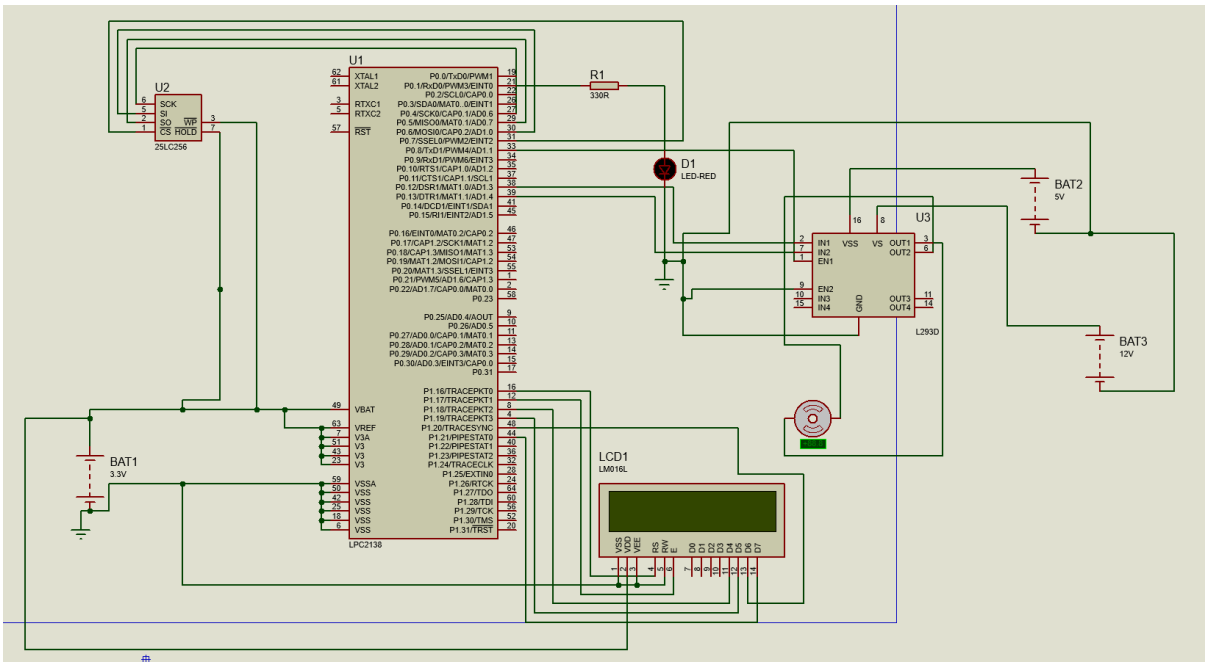


Figure 4.2: *Complete circuit in Proteus*

5. MOTOR PROFILES AND EEPROM STORAGE

5.1 The Six Profiles

Table 5.1: *Motor profiles*

No.	Name	Copies	What the motor does	Repeats	Time
1	DEMO	Test	Forward 75 % for 3 s, stop, reverse 40 % for 3 s, stop	1	9.6 s
2	WASH	Washing-machine wash	Forward 60 % 2 s, stop, reverse 60 % 2 s, stop	3	18.6 s
3	SPIN	Spin-dry	Start at 40 %, go up to 100 %, then slow stop	1	15.0 s
4	GATE	Gate / garage door	Open (forward 80 %), wait 3 s, close (reverse 80 %)	1	16.2 s
5	FAN	Breeze fan	30 → 60 → 90 → 50 %, 2 s each	2	24.0 s
6	MIXER	Blender pulse	Full speed 0.7 s, stop 0.5 s	3	4.8 s

The full sequence takes **88.2 s** and then starts again. In the program each profile is kept in simple arrays: `dir[profile][step]`, `speed[profile][step]`, `ramp[profile][step]` and `run[profile][step]`.

5.2 How a Profile Is Stored

Each profile is stored in its own 64-byte page of the EEPROM (profile 1 at address 0, profile 2 at 64, and so on). A page starts with a 12-byte header, then 6 bytes for each step, and ends with a checksum byte.

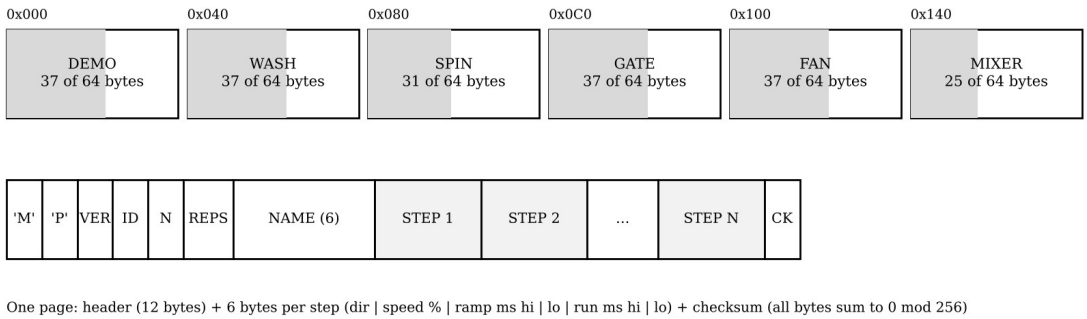


Figure 5.1: *EEPROM page map and the layout of one page*

Table 5.2: *Contents of one page*

Bytes	Content	Why
0 – 1	'M', 'P'	Marks a valid profile page
2	Version (3)	Detects data from an older program
3	Profile number	Detects a page in the wrong place
4, 5	Number of steps, repeats	—
6 – 11	Name (up to 6 letters)	Shown on the LCD
6 bytes per step	Direction, speed, ramp time (2 bytes), run time (2 bytes)	The step itself
Last byte	Checksum	Chosen so that all bytes add up to 0; any changed byte is detected

When the system starts, every page is read and checked. If a page is empty (a new chip reads all 0xFF) or damaged, the program writes the default profile into that page, reads it back and checks it again.

6. HOW THE SYSTEM WORKS

6.1 Step-by-Step Operation

1. The program sets the clocks, the pins, Timer1 and SPI.
2. The LCD shows "MOTOR PROFILES / Loading EEPROM" and the LED blinks three times.
3. The six profiles are read from the EEPROM and checked; bad pages are repaired.
4. Each profile is run for its number of repeats. For each step the motor first slows to zero if the direction changes, waits 200 ms before reversing, speeds up smoothly to the new speed, and then keeps that speed for the run time.
5. While waiting, the program keeps doing small background jobs: making the PWM signal, blinking the LED, checking the button and sending the next character to the LCD.
6. After the last profile it starts again with the first. The NEXT button skips to the next profile.

6.2 Exact Timing

Timer1 counts every 10 µs and never stops, so it works like a clock. The program keeps a variable `now` that holds the **planned** time of the next event. To hold a speed for 3 seconds, it adds 3 seconds to `now` and waits until the timer reaches that time. Because each new time is calculated from the planned time and not from "whenever the program got here", small delays never add up, and the whole 88.2 s sequence always takes exactly the same time.

6.3 Main Calculations

Table 6.1: *Main calculations*

Quantity	Calculation	Result
Processor clock	12 MHz crystal, PLL off	12 MHz
Peripheral clock	12 MHz ÷ 4	3 MHz
Timer1 count	3 MHz ÷ 30	every 10 µs
PWM	period 10 ms, steps of 1 %	100 Hz
SPI clock	3 MHz ÷ 8	375 kHz
Time to read all profiles	6 pages × 67 bytes × 21.3 µs	about 9 ms
Pause before reversing	fixed	200 ms
Button check	must stay pressed for	20 ms
LED current	(3.3 V − 1.8 V) ÷ 330 Ω	about 4.5 mA

6.4 LED Patterns and LCD Screen

Table 6.2: *LED patterns*

Motor state	LED
Starting up	3 quick blinks
Speeding up or slowing down	Fast blink (5 times a second)
Running forward	Slow blink (0.5 s on, 0.5 s off)
Running in reverse	Double flash every second
Stopped	One short flash every second
EEPROM fault	Stays on, motor stays off

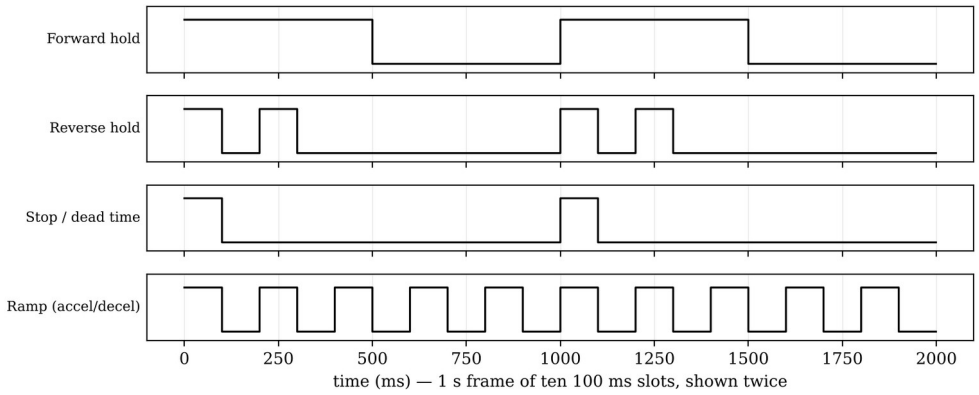


Figure 6.1: *LED patterns over two seconds*

Line 1 of the LCD shows the profile name, direction and target speed; line 2 shows the phase (ACCEL, HOLD, DECEL, DEAD or OFF), the step number and the repeat number, for example `WASH FWD 60% / HOLD S1/4 R2/3`.

7. SOFTWARE

7.1 Building the Program in Keil

1. Create a new project and choose the device NXP LPC2138 (or LPC2148).
2. Add `main.c` to the project.
3. In Options for Target, tick *Create HEX File*.
4. Build (F7). Load the .hex file into the LPC2138 in Proteus and set its Clock Frequency to 12 MHz.

7.2 Program Structure

The program is written in plain C using only arrays, `#define` constants and functions. It has 631 lines, written one statement per line.

Table 7.1: *Main functions*

Function(s)	What it does
`main()`	Sets up the chip, LCD and SPI, loads the profiles, then runs them forever
`load_profiles()`, `encode()`, `decode()`, `checksum()`	Reads, checks and repairs the profile pages
`spi()`, `ee_status()`, `ee_read_page()`, `ee_write_page()`	Talk to the EEPROM over SPI
`run_profile()`, `run_step()`	Run the steps of a profile
`ramp_to()`, `set_speed()`, `set_dir()`	Change the motor speed smoothly and set its direction safely
`service()`, `wait_until()`, `delay_ms()`	Waiting, with the background jobs: PWM, LED, button, LCD
`show_status()`, `lcd_text()`, `lcd_num()`, `lcd_byte()`	Prepare and send text to the LCD

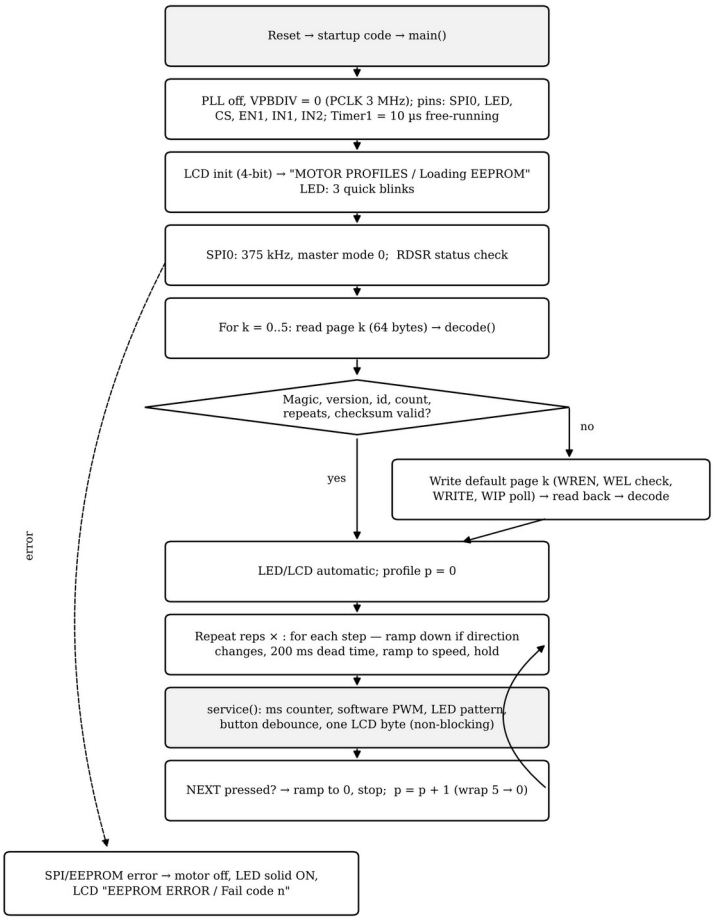


Figure 7.1: *Flowchart of the program*

7.3 Error Handling

If the EEPROM cannot be used, the motor is not started, the LED stays on and the LCD shows "EEPROM ERROR" with a code:

Table 7.2: *Error codes*

Code	Meaning
1	No reply on SPI (time-out)
2	SPI error
3	No EEPROM found
4	EEPROM is write-protected
5	EEPROM did not allow writing
6	EEPROM write did not finish
9	Profile still wrong after repair

8. SOURCE CODE

The complete program, `main.c` (631 lines):

Listing 8.1: main.c *(631 lines)*

```
/*
 * SPI EEPROM Motor-Profile Controller (LPC2138 / LPC2148)
 *
 * EEPROM 25LC256 : SCK P0.4, MISO P0.5, MOSI P0.6, CS P0.7
 * L293D          : EN1 P0.8, IN1 P0.12, IN2 P0.13
 * LED P0.1, NEXT button P0.15
 * LCD            : RS P1.16, E P1.17, D4-D7 P1.18-P1.21
 */

#define PINSEL0 (*(volatile unsigned int *)0xE002C000)
#define PINSEL2 (*(volatile unsigned int *)0xE002C014)
#define IO0PIN  (*(volatile unsigned int *)0xE0028000)
#define IO0SET  (*(volatile unsigned int *)0xE0028004)
#define IO0DIR  (*(volatile unsigned int *)0xE0028008)
#define IO0CLR  (*(volatile unsigned int *)0xE002800C)
#define IO1SET  (*(volatile unsigned int *)0xE0028014)
#define IO1DIR  (*(volatile unsigned int *)0xE0028018)
#define IO1CLR  (*(volatile unsigned int *)0xE002801C)
#define S0SPCR  (*(volatile unsigned int *)0xE0020000)
#define S0SPSR  (*(volatile unsigned int *)0xE0020004)
#define S0SPDR  (*(volatile unsigned int *)0xE0020008)
#define S0SPCCR (*(volatile unsigned int *)0xE002000C)
#define T1TCR   (*(volatile unsigned int *)0xE0008004)
#define T1TC    (*(volatile unsigned int *)0xE0008008)
#define T1PR    (*(volatile unsigned int *)0xE000800C)
#define PLLCON  (*(volatile unsigned int *)0xE01FC080)
#define PLLFEED (*(volatile unsigned int *)0xE01FC08C)
#define PCONP   (*(volatile unsigned int *)0xE01FC0C4)
#define VPBDIV  (*(volatile unsigned int *)0xE01FC100)
#define SCS     (*(volatile unsigned int *)0xE01FC1A0)

#define FOSC  12000000
#define MS    100           /* timer ticks every 10 us, so 100 ticks = 1 ms */

#define LED   (1 << 1)
#define CS    (1 << 7)
#define EN1   (1 << 8)
#define IN1   (1 << 12)
#define IN2   (1 << 13)

#define STOP  0
#define FWD   1
#define REV   2

#define DECEL 1
#define DEAD  2
#define ACCEL 3
#define HOLD  4

/* Default profiles: each step has direction, speed %, ramp ms, run ms */
const char def_name[6][7] = { "DEMO", "WASH", "SPIN", "GATE", "FAN", "MIXER" };
const unsigned int def_reps[6]  = { 1, 3, 1, 1, 2, 3 };
const unsigned int def_steps[6] = { 4, 4, 3, 4, 4, 2 };

const unsigned int def_dir[6][4] = {
    {FWD, STOP, REV,  STOP},
    {FWD, STOP, REV,  STOP},
    {FWD, FWD,  STOP, 0},
    {FWD, STOP, REV,  STOP},
    {FWD, FWD,  FWD,  FWD},
    {FWD, STOP, 0,    0}
};

const unsigned int def_speed[6][4] = {
    {75,  0,   40, 0},
    {60,  0,   60, 0},
    {40,  100, 0,  0},
    {80,  0,   80, 0},
    {30,  60,  90, 50},
    {100, 0,   0,  0}
};

const unsigned int def_ramp[6][4] = {
    {500,  300,  500,  300},
    {300,  300,  300,  300},
    {1000, 2000, 3000, 0},
    {800,  800,  800,  800},
    {1000, 1000, 1000, 1000},
    {200,  200,  0,    0}
};

```

```
const unsigned int def_run[6][4] = {
    {3000, 1000, 3000, 1000},
    {2000, 500,  2000, 500},
    {2000, 5000, 2000, 0},
    {4000, 3000, 4000, 2000},
    {2000, 2000, 2000, 2000},
    {700,  500,  0,    0}
};

const char dir_text[3][5]   = { "STOP", "FWD", "REV" };
const char phase_text[5][6] = { "IDLE", "DECEL", "DEAD", "ACCEL", "HOLD" };
const unsigned int pattern[4] = { 0x001, 0x01F, 0x005, 0x155 };   /* LED: stop, fwd, rev, ramp */

/* Profiles loaded from the EEPROM */
char name[6][7];
unsigned int reps[6], nsteps[6];
unsigned int dir[6][8], speed[6][8], ramp[6][8], run[6][8];

unsigned char buf[64];      /* one EEPROM page */
char lcd[32];               /* LCD text, 2 lines x 16 */

unsigned int now, duty, pwm_start, ms_mark;
unsigned int last_dir, off_since;
unsigned int running, skip, btn, btn_count;
unsigned int led_ms, led_slot, led_pat;
unsigned int lcd_left, lcd_time;
unsigned int spi_err, fail;
unsigned int profile, rep, step, cur_dir, phase, cycles;

void pin(unsigned int mask, int on) {
    if (on)
        IO0SET = mask;
    else
        IO0CLR = mask;
}

int reached(unsigned int t) {
    return (int)(T1TC - t) >= 0;    /* safe across timer overflow */
}

/* ---------------- LCD (4-bit) ---------------- */

void lcd_nibble(int rs, int v) {
    unsigned int t;

    IO1CLR = 0x3F << 16;
    IO1SET = (rs << 16) | ((v & 15) << 18) | (1 << 17);    /* data out, E high */
    t = T1TC;
    while (T1TC - t < 2);
    IO1CLR = 1 << 17;                                      /* E low, LCD latches */
}

void lcd_byte(int rs, int v) {
    lcd_nibble(rs, v >> 4);
    lcd_nibble(rs, v);
}

void lcd_clear(void) {
    int i;

    for (i = 0; i < 32; i++)
        lcd[i] = ' ';
}

void lcd_text(int pos, const char s[]) {
    int i;

    for (i = 0; s[i] != 0; i++)
        lcd[pos + i] = s[i];
}

void lcd_num(int pos, int w, unsigned int v) {   /* right-aligned number */
    do {
        w--;
        lcd[pos + w] = '0' + v % 10;
        v = v / 10;
    } while (v > 0 && w > 0);
}

/* ---------------- Background jobs (called inside every wait) ---------------- */

void service(void) {
    unsigned int t = T1TC;
    unsigned int p;

    while (t - ms_mark >= MS) {             /* 1 ms tick */
        ms_mark += MS;

        if (++led_ms == 100) {
```

```
            led_ms = 0;
            if (++led_slot == 10)
                led_slot = 0;
        }

        p = (IO0PIN >> 15) & 1;             /* button is active low, 20 ms debounce */
        if (running == 0 || p == btn) {
            btn_count = 0;
        } else if (++btn_count >= 20) {
            btn = p;
            btn_count = 0;
            if (p == 0)
                skip = 1;
        }
    }

    while (t - pwm_start >= 10 * MS)
        pwm_start += 10 * MS;
    pin(EN1, t - pwm_start < duty * 10);    /* software PWM, 10 ms period */

    if (running) {
        if (phase == ACCEL || phase == DECEL)
            p = 3;
        else
            p = cur_dir;

        if (p + 1 != led_pat) {
            led_pat = p + 1;
            led_ms = 0;
            led_slot = 0;
        }
        pin(LED, (pattern[p] >> led_slot) & 1);
    }

    if (lcd_left > 0 && t - lcd_time >= 5) {   /* one LCD byte every 50 us */
        p = 34 - lcd_left;
        lcd_left--;

        if (p == 0)
            lcd_byte(0, 0x80);
        else if (p == 17)
            lcd_byte(0, 0xC0);
        else if (p < 17)
            lcd_byte(1, lcd[p - 1]);
        else
            lcd_byte(1, lcd[p - 2]);

        lcd_time = T1TC;
    }
}

int wait_until(unsigned int t) {            /* returns 0 if NEXT was pressed */
    while (!reached(t)) {
        if (skip)
            return 0;
        service();
    }
    return 1;
}

void delay_ms(unsigned int ms) {
    unsigned int t = T1TC + ms * MS;

    while (!reached(t))
        service();
}

void show_status(unsigned int target) {     /* e.g. "WASH FWD 60%" */
    lcd_clear();
    lcd_text(0, name[profile]);
    lcd_text(7, dir_text[cur_dir]);
    lcd_num(11, 3, target);
    lcd_text(14, "%");

    if (cur_dir == STOP && phase == HOLD)
        lcd_text(16, "OFF");
    else
        lcd_text(16, phase_text[phase]);

    lcd_text(22, "S /   R /");
    lcd_num(23, 1, step + 1);
    lcd_num(25, 1, nsteps[profile]);
    lcd_num(29, 1, rep + 1);
    lcd_num(31, 1, reps[profile]);
    lcd_left = 34;
}

/* ---------------- SPI and 25LC256 EEPROM ---------------- */

```

```
unsigned char spi(unsigned char tx) {
    unsigned int timeout = 100000;
    unsigned int s;

    if (spi_err)
        return 0;

    S0SPDR = tx;
    do {
        s = S0SPSR;
        if (s & 0x18)
            spi_err = 2;                    /* mode fault or abort */
        if (--timeout == 0)
            spi_err = 1;                    /* no response */
    } while ((s & 0x80) == 0 && spi_err == 0);

    return S0SPDR;                          /* reading it clears the done flag */
}

unsigned char ee_status(void) {
    unsigned char s;

    IO0CLR = CS;
    spi(0x05);                              /* RDSR */
    s = spi(0);
    IO0SET = CS;
    return s;
}

void ee_read_page(unsigned int addr) {      /* 64 bytes into buf[] */
    int i;

    IO0CLR = CS;
    spi(0x03);                              /* READ */
    spi(addr >> 8);
    spi(addr & 0xFF);
    for (i = 0; i < 64; i++)
        buf[i] = spi(0);
    IO0SET = CS;
}

unsigned int ee_write_page(unsigned int addr, int n) {
    int i;
    unsigned char s;

    IO0CLR = CS;
    spi(0x06);                              /* WREN */
    IO0SET = CS;

    s = ee_status();
    if (spi_err)
        return spi_err;
    if ((s & 0x02) == 0)
        return 5;                           /* write enable not set */

    IO0CLR = CS;
    spi(0x02);                              /* WRITE */
    spi(addr >> 8);
    spi(addr & 0xFF);
    for (i = 0; i < n; i++)
        spi(buf[i]);
    IO0SET = CS;

    for (i = 0; i < 20000; i++) {           /* wait while write is in progress */
        s = ee_status();
        if (spi_err)
            return spi_err;
        if ((s & 0x01) == 0)
            return 0;
    }
    return 6;
}

/* ---------------- Profile page ----------------
   'M' 'P' 3 id steps reps name[6], then 6 bytes per step, then checksum */

unsigned char checksum(int n) {
    unsigned char s = 0;
    int i;

    for (i = 0; i < n; i++)
        s += buf[i];
    return s;
}

int encode(int k) {                         /* default profile k into buf[] */
    int i;
    int n = 12;

```

```
    buf[0] = 'M';
    buf[1] = 'P';
    buf[2] = 3;
    buf[3] = k;
    buf[4] = def_steps[k];
    buf[5] = def_reps[k];

    for (i = 0; i < 6; i++)
        buf[6 + i] = def_name[k][i];

    for (i = 0; i < (int)def_steps[k]; i++) {
        buf[n]     = def_dir[k][i];
        buf[n + 1] = def_speed[k][i];
        buf[n + 2] = def_ramp[k][i] >> 8;
        buf[n + 3] = def_ramp[k][i] & 0xFF;
        buf[n + 4] = def_run[k][i] >> 8;
        buf[n + 5] = def_run[k][i] & 0xFF;
        n += 6;
    }

    buf[n] = 0 - checksum(n);               /* whole record now sums to 0 */
    return n + 1;
}

int decode(int k) {                         /* buf[] into profile k, returns 1 if valid */
    int i;
    int n = buf[4];

    if (buf[0] != 'M' || buf[1] != 'P' || buf[2] != 3 || buf[3] != k)
        return 0;
    if (n < 1 || n > 8 || buf[5] < 1 || buf[5] > 9)
        return 0;
    if (checksum(13 + 6 * n) != 0)
        return 0;

    for (i = 0; i < n; i++) {
        if (buf[12 + 6 * i] > REV || buf[13 + 6 * i] > 100)
            return 0;
    }

    for (i = 0; i < n; i++) {
        dir[k][i]   = buf[12 + 6 * i];
        speed[k][i] = buf[13 + 6 * i];
        ramp[k][i]  = buf[14 + 6 * i] * 256 + buf[15 + 6 * i];
        run[k][i]   = buf[16 + 6 * i] * 256 + buf[17 + 6 * i];
    }

    for (i = 0; i < 6; i++)
        name[k][i] = buf[6 + i];
    name[k][6] = 0;
    nsteps[k] = n;
    reps[k] = buf[5];
    return 1;
}

unsigned int load_profiles(void) {
    int k;
    unsigned int e;
    unsigned char s = ee_status();

    if (spi_err)
        return spi_err;
    if (s == 0xFF)
        return 3;                           /* no EEPROM */
    if ((s & 0x0C) == 0x0C)
        return 4;                           /* write-protected */

    for (k = 0; k < 6; k++) {
        ee_read_page(k * 64);
        if (spi_err)
            return spi_err;
        if (decode(k))
            continue;

        /* bad page: write the default and read it back */
        e = ee_write_page(k * 64, encode(k));
        if (e != 0)
            return e;

        ee_read_page(k * 64);
        if (spi_err)
            return spi_err;
        if (!decode(k))
            return 9;
    }
    return 0;
}

/* ---------------- Motor ---------------- */
```

```
438
439 void set_speed(unsigned int p) {
440     duty = p;
441     service();
442 }
443
444 void set_dir(unsigned int d) {
445     IO0CLR = IN1 | IN2;                     /* IN1 and IN2 never high together */
446     if (d == FWD)
447         IO0SET = IN1;
448     if (d == REV)
449         IO0SET = IN2;
450
451     if (d == STOP && cur_dir != STOP) {
452         last_dir = cur_dir;
453         off_since = now;
454     }
455     cur_dir = d;
456 }
457
458 int ramp_to(unsigned int target, unsigned int ms) {   /* 1% steps, exactly ms long */
459     unsigned int from = duty;
460     unsigned int start = now;
461     unsigned int n, k;
462
463     if (target > from)
464         n = target - from;
465     else
466         n = from - target;
467
468     if (n == 0)
469         return 1;
470
471     for (k = 1; k <= n; k++) {
472         if (!wait_until(start + ms * MS * k / n))
473             return 0;
474
475         if (target > from)
476             set_speed(from + k);
477         else
478             set_speed(from - k);
479     }
480
481     now = start + ms * MS;
482     return 1;
483 }
484
485 void set_phase(unsigned int ph, unsigned int target) {
486     phase = ph;
487     show_status(target);
488 }
489
490 void run_step(void) {
491     unsigned int d = dir[profile][step];
492
493     if (cur_dir != STOP && d != cur_dir) {  /* slow down before changing direction */
494         set_phase(DECEL, 0);
495         if (!ramp_to(0, ramp[profile][step]))
496             return;
497         set_dir(STOP);
498     }
499
500     if (d != STOP && d != cur_dir) {
501         if (last_dir != STOP && last_dir != d && (int)(off_since + 200 * MS - now) > 0) {
502             set_phase(DEAD, 0);             /* 200 ms off before reversing */
503             now = off_since + 200 * MS;
504             if (!wait_until(now))
505                 return;
506         }
507         set_dir(d);
508     }
509
510     if (d != STOP) {
511         set_phase(ACCEL, speed[profile][step]);
512         if (!ramp_to(speed[profile][step], ramp[profile][step]))
513             return;
514     }
515
516     set_phase(HOLD, duty);
517     now += run[profile][step] * MS;
518     wait_until(now);
519 }
520
521 void run_profile(void) {
522     for (rep = 0; rep < reps[profile]; rep++) {
523         for (step = 0; step < nsteps[profile]; step++) {
524             run_step();
525             if (skip)
526                 return;
```

```
        }
    }
}

/* ---------------- Main ---------------- */

int main(void) {
    int i;

    PLLCON = 0;                             /* PLL off, CPU at 12 MHz */
    PLLFEED = 0xAA;
    PLLFEED = 0x55;
    VPBDIV = 0;                             /* peripherals at 3 MHz */
    SCS = 0;

    PINSEL0 = (PINSEL0 & 0x30FC00F3) | 0x1500;   /* P0.4-P0.6 = SPI, rest GPIO */
    PINSEL2 = PINSEL2 & ~8;                      /* P1.16-P1.25 = GPIO */

    IO0SET = CS;
    IO0CLR = LED | EN1 | IN1 | IN2;
    IO0DIR |= LED | CS | EN1 | IN1 | IN2;
    IO1CLR = 0x3F << 16;
    IO1DIR |= 0x3F << 16;

    PCONP |= (1 << 2) | (1 << 8);           /* Timer1 and SPI0 on */

    T1TCR = 2;
    T1PR = FOSC / 4 / 100000 - 1;           /* Timer1 tick = 10 us */
    T1TCR = 1;
    ms_mark = T1TC;
    pwm_start = ms_mark;

    S0SPCCR = 8;
    S0SPCR = 0x20;                          /* SPI master, mode 0, 375 kHz */

    /* LCD init: 3, 3, 3, 2, then settings */
    delay_ms(40);
    for (i = 0; i < 4; i++) {
        if (i < 3)
            lcd_nibble(0, 3);
        else
            lcd_nibble(0, 2);

        if (i == 0)
            delay_ms(5);
        else
            delay_ms(1);
    }
    lcd_byte(0, 0x28);
    delay_ms(1);
    lcd_byte(0, 0x0C);
    delay_ms(1);
    lcd_byte(0, 0x06);
    delay_ms(1);
    lcd_byte(0, 0x01);
    delay_ms(2);

    lcd_clear();
    lcd_text(0, "MOTOR PROFILES");
    lcd_text(16, "Loading EEPROM");
    lcd_left = 34;

    for (i = 0; i < 3; i++) {
        pin(LED, 1);
        delay_ms(100);
        pin(LED, 0);
        delay_ms(100);
    }

    fail = load_profiles();
    if (fail) {                             /* EEPROM fault: LED on, motor off */
        lcd_clear();
        lcd_text(0, "EEPROM ERROR");
        lcd_text(16, "Fail code");
        lcd_num(26, 1, fail);
        lcd_left = 34;
        pin(LED, 1);
        while (1)
            service();
    }

    btn = (IO0PIN >> 15) & 1;
    running = 1;
    now = T1TC;

    while (1) {
        if (!skip)
            run_profile();

```

```
616         if (skip) {                         /* NEXT: ramp down, stop, go to next profile */
617             skip = 0;
618             now = T1TC;
619             set_phase(DECEL, 0);
620             ramp_to(0, 300);
621             set_speed(0);
622             now = T1TC;
623             set_dir(STOP);
624         }
625
626         if (++profile == 6) {
627             profile = 0;
628             cycles++;
629         }
630     }
631 }
```

9. RESULTS

9.1 Program Build and EEPROM Contents

```
linking...
Program Size: Code=6488 RO-data=452 RW-data=108 ZI-data=1700
FromELF: creating hex file...
".\Objects\sp_mot.axf" - 0 Error(s), 8 Warning(s).
```

Figure 9.1: *Keil µVision build output*

On the first run the EEPROM was empty, so the program wrote the six default profiles. Figure 9.2 shows the EEPROM memory in Proteus. The bytes match exactly what the program should write. For example, page 0 starts with `4D 50 03 00 04 01 44 45 4D 4F` which means 'M', 'P', version 3, profile 0, 4 steps, 1 repeat, "DEMO".

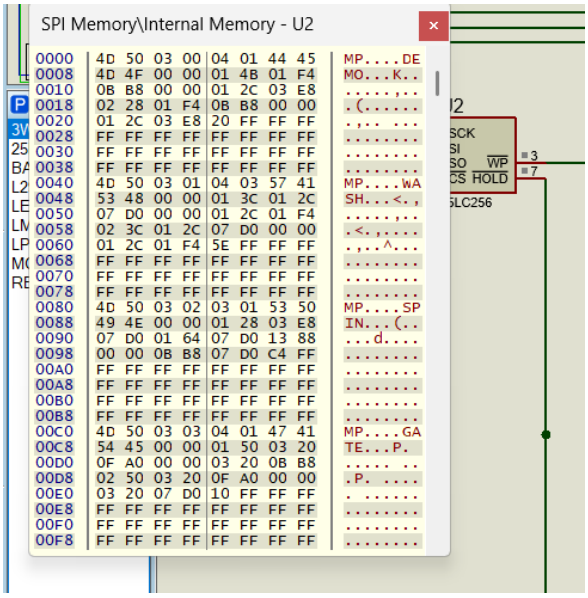
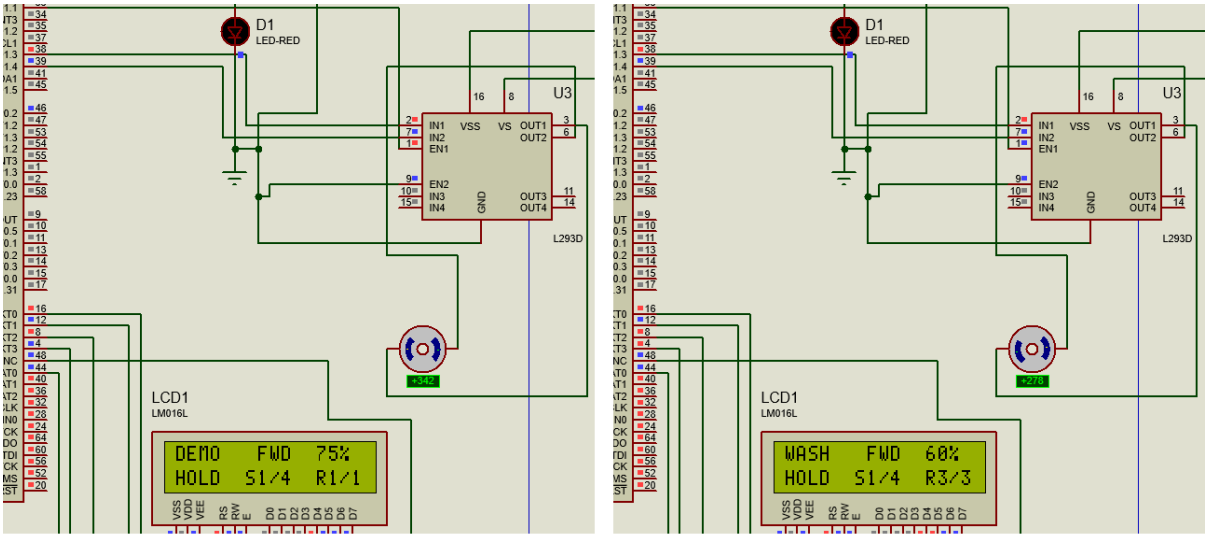


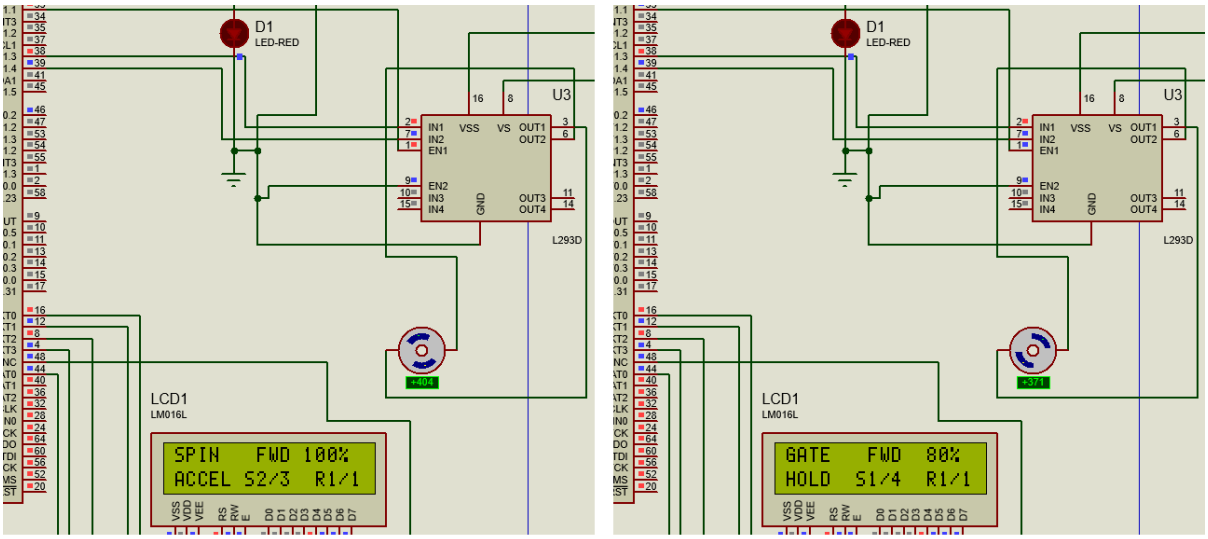
Figure 9.2: *EEPROM contents in Proteus after the first run*

9.2 Profiles Running in Proteus



(a) DEMO, forward hold (b) WASH, repeat 3 of 3

Figure 9.3: *DEMO running forward (342 RPM) and WASH on its third repeat (278 RPM)*



(a) SPIN, ramp to 100 % (b) GATE, opening

Figure 9.4: *SPIN speeding up to 100 % (404 RPM) and GATE opening (371 RPM)*

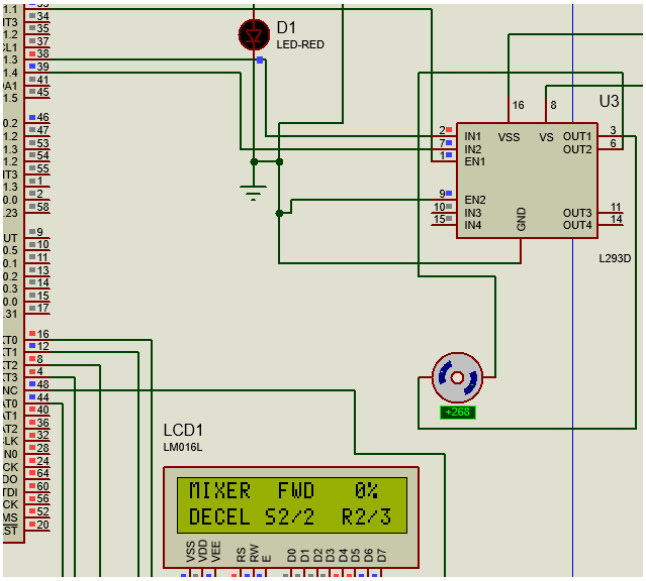


Figure 9.5: *MIXER slowing down after a pulse (268 RPM)*

Table 9.1: *Motor speed seen in Proteus*

Profile and step	PWM duty	Motor speed
DEMO, forward	75 %	342 RPM
WASH, forward	60 %	278 RPM
GATE, opening	80 %	371 RPM
SPIN, speeding up	rising	404 RPM and rising

Higher duty gives higher speed, as expected. In every screenshot the LCD, the L293D inputs and the direction of the motor agree.

9.3 LED and LCD at Different Times

Table 9.2 shows what the LED and LCD show during the whole sequence (times from switch-on, 12 MHz clock). Repeated parts are written once.

Table 9.2: *LED and LCD status during the full sequence*

Profile (time)	LED	LCD
Start-up 0 – 0.7 s	3 quick blinks	`MOTOR PROFILES` `Loading EEPROM`
DEMO 0.7 – 1.2 s	Fast blink (5 times a second)	`DEMO  FWD  75%` `ACCEL S1/4  R1/1`
DEMO 1.2 – 4.2 s	Slow blink (0.5 s on, 0.5 s off)	`DEMO  FWD  75%` `HOLD  S1/4  R1/1`
DEMO 4.2 – 4.5 s	Fast blink (5 times a second)	`DEMO  FWD  0%` `DECEL S2/4  R1/1`
DEMO 4.5 – 5.5 s	One short flash every second	`DEMO  STOP  0%` `OFF  S2/4  R1/1`
DEMO 5.5 – 6.0 s	Fast blink (5 times a second)	`DEMO  REV  40%` `ACCEL S3/4  R1/1`
DEMO 6.0 – 9.0 s	Double flash every second	`DEMO  REV  40%` `HOLD  S3/4  R1/1`
DEMO 9.0 – 9.3 s	Fast blink (5 times a second)	`DEMO  REV  0%` `DECEL S4/4  R1/1`
DEMO 9.3 – 10.3 s	One short flash every second	`DEMO  STOP  0%` `OFF  S4/4  R1/1`
WASH 10.3 – 10.6 s	Fast blink (5 times a second)	`WASH  FWD  60%` `ACCEL S1/4  R1/3`
WASH 10.6 – 12.6 s	Slow blink (0.5 s on, 0.5 s off)	`WASH  FWD  60%` `HOLD  S1/4  R1/3`
WASH 12.6 – 12.9 s	Fast blink (5 times a second)	`WASH  FWD  0%` `DECEL S2/4  R1/3`
WASH 12.9 – 13.4 s	One short flash every second	`WASH  STOP  0%` `OFF  S2/4  R1/3`
WASH 13.4 – 13.7 s	Fast blink (5 times a second)	`WASH  REV  60%` `ACCEL S3/4  R1/3`
WASH 13.7 – 15.7 s	Double flash every second	`WASH  REV  60%` `HOLD  S3/4  R1/3`
WASH 15.7 – 16.5 s	Fast blink, then one flash	`WASH  STOP  0%` `OFF  S4/4  R1/3`
WASH (repeats 2, 3) 16.5 – 28.9 s	Same as above	Same, with `R2/3`, `R3/3`
SPIN 28.9 – 29.9 s	Fast blink (5 times a second)	`SPIN  FWD  40%` `ACCEL S1/3  R1/1`
SPIN 29.9 – 31.9 s	Slow blink (0.5 s on, 0.5 s off)	`SPIN  FWD  40%` `HOLD  S1/3  R1/1`
SPIN 31.9 – 33.9 s	Fast blink (5 times a second)	`SPIN  FWD 100%` `ACCEL S2/3  R1/1`
SPIN 33.9 – 38.9 s	Slow blink (0.5 s on, 0.5 s off)	`SPIN  FWD 100%` `HOLD  S2/3  R1/1`
SPIN 38.9 – 41.9 s	Fast blink (5 times a second)	`SPIN  FWD  0%` `DECEL S3/3  R1/1`
SPIN 41.9 – 43.9 s	One short flash every second	`SPIN  STOP  0%` `OFF  S3/3  R1/1`

Profile (time)	LED	LCD
GATE 43.9 – 44.7 s	Fast blink (5 times a second)	`GATE  FWD  80%` `ACCEL S1/4  R1/1`
GATE 44.7 – 48.7 s	Slow blink (0.5 s on, 0.5 s off)	`GATE  FWD  80%` `HOLD  S1/4  R1/1`
GATE 48.7 – 52.5 s	Fast blink, then one flash	`GATE  STOP  0%` `OFF  S2/4  R1/1`
GATE 52.5 – 53.3 s	Fast blink (5 times a second)	`GATE  REV  80%` `ACCEL S3/4  R1/1`
GATE 53.3 – 57.3 s	Double flash every second	`GATE  REV  80%` `HOLD  S3/4  R1/1`
GATE 57.3 – 60.1 s	Fast blink, then one flash	`GATE  STOP  0%` `OFF  S4/4  R1/1`

FAN
60.1 – 72.1 s
	Fast blink on each change, slow blink while steady	`FAN  FWD  30%`
		`HOLD  S1/4  R1/2`
		… 60%, 90%, 50%

FAN (repeat 2) 72.1 – 84.1 s	Same as above	Same, with `R2/2`
MIXER 84.1 – 85.0 s	Fast blink, then slow blink	`MIXER  FWD 100%` `HOLD  S1/2  R1/3`
MIXER 85.0 – 85.7 s	Fast blink, then one flash	`MIXER  STOP  0%` `OFF  S2/2  R1/3`
MIXER (repeats 2, 3) 85.7 – 88.9 s	Same as above	Same, with `R2/3`, `R3/3`
DEMO again 88.9 → s	Sequence repeats	`DEMO  FWD  75%` `ACCEL S1/4  R1/1`

9.4 Timing Check

The program was also tested in a computer model of the ARM chip with its SPI, timer, EEPROM and LCD. Over the full sequence it gave the expected results:

Table 9.3: *Test results*

Check	Result
Full sequence time	88.2 s, then back to DEMO
PWM at 75 % and 40 %	7.50 ms and 4.00 ms on-time in every 10 ms (error under 0.03 ms)
Pause before every reversal	always more than 200 ms
IN1 and IN2 high together	never
NEXT button	each press skips exactly one profile
EEPROM faults (no EEPROM, broken data line, no SPI clock)	error code shown, motor stays off

9.5 Problems Faced and Solutions

Table 9.4: *Problems and solutions*

Problem	Solution
The motor speed in Proteus changed seconds after the LCD	The motor model was too heavy; its *Effective Mass* was reduced
All times were 1.2 times too long	The chip's clock in Proteus was 10 MHz; it was set to 12 MHz
Updating the LCD delayed the motor timing	The LCD is now sent one character at a time in the background
Keil's start-up code turned on the PLL (60 MHz)	The program switches the PLL off before setting the timer
Pressing the button once could skip several profiles	The button must stay pressed for 20 ms before it counts

10. APPLICATIONS AND FUTURE SCOPE

10.1 Applications

- Washing machines (wash and spin cycles)
- Automatic gates, garage doors and window blinds
- Fans with breeze modes
- Mixers and blenders
- Conveyor belts and other machines with repeated motor patterns

10.2 Future Scope

- Add a speed sensor so the controller can keep the exact speed (closed-loop control).
- Let the user change profiles from a keypad or a computer.
- Use the chip's hardware PWM at a higher frequency for a quieter motor.
- Build and test the circuit on real hardware.

11. CONCLUSION

An SPI EEPROM motor-profile controller was designed with the LPC2138, programmed in C in Keil µVision and tested in Proteus. Six motor profiles that copy everyday machines are stored safely in a 25LC256 EEPROM — one profile per page, with a checksum — and are repaired automatically if they are missing or damaged. The motor is driven through an L293D with PWM speed control, smooth speed changes and a pause before every reversal. A 16×2 LCD and LED blink patterns show the state at all times. The EEPROM contents in Proteus matched the program exactly, the motor followed every profile, and the timing stayed exact over the whole sequence. The next steps are speed feedback and testing on real hardware.

REFERENCES

[1] NXP Semiconductors, *UM10120: LPC2131/2/4/6/8 User Manual*.

[2] Microchip Technology Inc., *25AA256/25LC256 256K SPI Bus Serial EEPROM* data sheet.

[3] Texas Instruments, *L293x Quadruple Half-H Drivers* data sheet.

[4] Hitachi, *HD44780U Dot Matrix LCD Controller/Driver* data sheet.

[5] Keil, *µVision IDE* user guide.

[6] Labcenter Electronics, *Proteus Design Suite* documentation.